

Building a Neural Network from Scratch

Pratham Kelkar

July 2026

Contents

1	Motivation	3
2	Introduction to Neural Networks	3
2.1	The Problem to be Solved	3
2.2	Binary Classification	4
3	Logistic Regression	4
3.1	Mathematical Representation	4
3.1.1	Activation Functions	4
3.1.2	Loss and Cost Function	5
3.1.3	Learning Rate	5
3.2	Algorithm Structure	6
3.3	Computation of Derivatives	6
3.3.1	Backpropagation	6
3.4	Logistic Regression With Multiple Training Examples	8
3.4.1	Vectorization	8
3.4.2	Algorithm Structure for Multiple Training Examples	9
4	Neural Network Structure	10
4.1	Shallow Neural Network	10
4.1.1	Architecture	10
4.1.2	Backpropagation	12
4.1.3	Algorithm	14
4.2	Deep Neural Networks	14
5	Implementation	15
5.1	Familiarization With the Dataset	15
5.2	Pre-processing the Data	16
5.3	Initialization of Parameters	16
5.4	Code Structure	16
5.5	Testing	17
5.5.1	Logistic Regression on a Single Neuron	17
5.6	Deep Neural Network	18
5.7	Hyperparameter Tuning	18
5.7.1	Learning Rate	18
5.7.2	Model Architecture	19
5.8	Evaluation	22
6	Conclusion	22
7	Works Cited	23
8	Appendix	24

1 Motivation

Neural networks form the backbone of many technical applications we use today from FaceID on unlocking our phones to talking to artificial intelligence assistants like Chat-GPT and Gemini.

I have always been fascinated by how they work under the hood. Before this project, I knew that neural networks take in an input and perform some sort of multi-variable regression or classification on the input to provide an output. Although I understood the underlying mechanism that they work on, I did not know the rigorous mathematical concepts that they operate on and how one is actually implemented without the help of frameworks like TensorFlow and PyTorch.

Previously, I have experimented with these frameworks and created models by piling on dense layers and using various activation functions, but I never understood the underlying concepts or the implementation details. This is what I aim to develop through this project, hoping that it will leave me in a better position to create and optimize models in the future.

For this project, I will be using the ATLAS Higgs Boson Challenge dataset, originally released by CERN and hosted through a Kaggle competition (“Higgs Boson Machine Learning Challenge”), to train and test my models. This makes it a good fit for training and testing, since it has 250,000 rows of data spanning 30 features, along with the fact that it has been widely used and benchmarked before, giving me a good point of reference to compare my own results against.

2 Introduction to Neural Networks

Neural networks are essentially machine learning algorithms used to predict the output of a given input by performing mathematical operations on it. Mathematically, a neural network learns a function $f(\mathbf{x})$ by mapping an input vector $\mathbf{x} = [x_1, x_2, x_3, \dots, x_n]$ to predict a response $\mathbf{y}$.

The simplest form of a neural network is a linear regression model, which can be thought of as a neural network with a single neuron in it. Essentially, all neurons of a neural network perform a linear transformation on the input variable to provide an output that minimizes the overall loss of the predictions it makes.

2.1 The Problem to be Solved

Many neural networks try to solve classification problems (for example: Google Image Search or FaceID on a phone). They process inputs and “classify” them into one of many (or in some cases just two) bins. I will be focusing on the framework for binary classification problems, which can then be modified to fit multi-variable classification.

2.2 Binary Classification

The binary classification problem focuses on classifying an input $\mathbf{x}$ as a 1 (True) or a 0 (False) as a prediction $\hat{y}$ made by the model. For example, if we take up the problem of identifying whether an image contains a cat, the model will output a 1 for images that it predicts contains a cat and a 0 for images that it predicts to not contain a cat. It processes an input with a set of features to predict the probability that the given input is True or False. If this probability exceeds a numerical threshold ϵ , then it can be classified as a 1.

In mathematical terms, given an input $\mathbf{x}$, we find

$$\hat{y} = P(y = 1|\mathbf{x})$$

The simplest way to build such a neural network is by understanding the logistic regression algorithm as a base for the process that each neuron executes.

3 Logistic Regression

3.1 Mathematical Representation

Logistic regression is a simple algorithm that applies a linear transformation to an input and then converts it into a probability using the sigmoid function. It repeats this process for multiple iterations to minimize the error or the "loss" from each prediction.

Taking $\mathbf{x} = [x_1, x_2, \dots, x_n]$, $\mathbf{x} \in \mathbb{R}^n$ as an input vector with a set of features, we want to find the output $\hat{y}$ such that:

$$\hat{y} = P(y = 1|\mathbf{x})$$

Here, we apply a linear transformation of the form $y = mx + c$ to $\mathbf{x}$, using the parameters $\mathbf{w} = [w_1, w_2, \dots, w_n]$ and b , such that $\mathbf{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$. This can be represented as:

$$z = \mathbf{w}^T \mathbf{x} + b \tag{1}$$

3.1.1 Activation Functions

Now that we have the linear part z , we need to turn it into a prediction. On its own, the value $z = \mathbf{w}^T \mathbf{x} + b$ is unbounded and can take any value in $\mathbb{R}$, so it cannot be read directly as a probability. To fix this, we pass z through a non-linear function called an "activation function", which is written as $g(\cdot)$, so that the output of a neuron is $a = g(z)$.

An activation function serves two purposes. First, it maps the unbounded output of the linear step into a useful range. Second, and more importantly for the more complicated networks with multiple layers, it introduces non-linearity into the model: if we only composed linear transformations, then stacking many neurons on top of one another would still collapse into a single linear transformation, regardless of how many were used, and the model could never capture the complex, curved patterns that real data demands. The activation function is what breaks this linearity. For logistic regression, the activation we choose is the sigmoid function, defined as:

$$a = g(z) = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

The sigmoid squashes any real input into the open interval $(0, 1)$, which is exactly what we need in order to interpret the output as the probability that the input belongs to the positive class. Thus,

$$a = \sigma(z) = \hat{y}$$

3.1.2 Loss and Cost Function

The error in the prediction can be represented by:

$$y - \hat{y}$$

Thus, we can now define a cost function that we can minimize to ensure that our predictions keep getting more accurate with each iteration of this process.

For the loss function, the squared error function $(\hat{y} - y)^2$ is not used because it leads to a non-convex optimization problem (being a function of $\mathbf{w}$ and b) with multiple optima where the algorithm could get stuck, and thus the global minima may not be found (Ng). Instead, we can define the loss function as follows:

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log (1 - \hat{y})) \quad (3)$$

Here, if $y = 1$, $\mathcal{L}(\hat{y}, y) = -\log \hat{y}$ and if $y = 0$, $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$.

Now, taking $\hat{y} = a$, the cost function for 1 training example can be defined as:

$$J(\mathbf{w}, b) = \mathcal{L}(\hat{y}, y) = -[y \log a + (1 - y) \log (1 - a)] \quad (4)$$

Now that the cost function $J(\mathbf{w}, b)$ is defined, it needs to be minimized to train the model on the set of m training examples so that it can accurately classify them. This can be done by taking the gradient descent step. This is a small step to update the parameters $\mathbf{w}$ and b in the direction of steepest gradient of the cost function. This is a two-step process represented by:

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}} = \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} J(\mathbf{w}, b) \quad (5)$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b} = b - \alpha \nabla_b J(\mathbf{w}, b) \quad (6)$$

3.1.3 Learning Rate

The parameter α appearing in equations (5) and (6) is called the “learning rate”. The gradient only tells us the direction in which the cost J increases most steeply, so moving in the opposite direction decreases it; α is what controls how large a step is taken in that direction on each iteration.

Choosing α well matters. If it is too large, each update can overshoot the minimum of J , causing the cost to oscillate or even diverge instead of settling down. If it is too small, each step barely moves $\mathbf{w}$ and b , and the algorithm needs many more iterations to converge, making training slow. Unlike $\mathbf{w}$ and b , α is not learned from the data: it is fixed before training begins, and choosing a suitable value is part of designing the algorithm rather than something gradient descent solves for on its own.

3.2 Algorithm Structure

Thus, using equations (1), (2), (3), (4), (5), and (6) for p iterations and 1 training example, the algorithm can be represented as:

Repeat, for $k = 1$ to p : {

$$\begin{aligned} z &= \mathbf{w}^T \mathbf{x} + b \\ a &= \hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \\ J(\mathbf{w}, b) &= -[y \log a + (1 - y) \log (1 - a)] \\ \mathbf{w} &= \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}} = \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} J(\mathbf{w}, b) \\ b &= b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b} = b - \alpha \nabla_b J(\mathbf{w}, b) \end{aligned}$$

}

Note here that repeating the above algorithm multiple times will have no effect as there is only 1 training example, but this idea will be generalized with multiple training examples later.

3.3 Computation of Derivatives

To minimize the cost function $J(\mathbf{w}, b)$, we must compute its partial derivatives with respect to the two variables $\mathbf{w}$ and b . To do so, we can set up a computation graph to calculate the derivatives using the chain rule of calculus.

3.3.1 Backpropagation

Backpropagation is the procedure we use to compute these derivatives efficiently. The idea is to read the computation graph from right to left: starting from the cost J at the end, we move backwards one operation at a time, and at each stage we use the chain rule to turn the derivative we have already found into the derivative of the next variable further back. Because every stage reuses the result of the stage before it, we obtain the derivative of the cost with respect to every parameter in a single backward pass, rather than differentiating each parameter from scratch.

Before we begin, it is worth introducing a piece of shorthand that will keep the derivations that follow compact. Every derivative we care about is the derivative of the final quantity, the cost J (which for a single training example is just $\mathcal{L}$), with respect to some intermediate variable. Rather than writing $\frac{\partial J}{\partial z}$, $\frac{\partial J}{\partial \mathbf{w}}$, and $\frac{\partial J}{\partial b}$ out in full every time, we adopt the convention

$$d\text{var} \equiv \frac{\partial J}{\partial \text{var}},$$

so that $dz = \frac{\partial J}{\partial z}$, $d\mathbf{w} = \frac{\partial J}{\partial \mathbf{w}}$, $db = \frac{\partial J}{\partial b}$, and so on. The prefix d is read as “the gradient of the cost with respect to”, and it matches the variable names we will later use in code.

$$\mathbf{w} \rightarrow z \rightarrow a \rightarrow J$$

We must compute $\frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$ and $\frac{\partial J(\mathbf{w}, b)}{\partial b}$. Using the chain rule, we can write:

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial a} \times \frac{\partial a}{\partial z} \quad (7)$$

Similarly,

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{\partial J}{\partial z} \times \frac{\partial z}{\partial \mathbf{w}}, \quad \frac{\partial J}{\partial b} = \frac{\partial J}{\partial z} \times \frac{\partial z}{\partial b} \quad (8)$$

Thus, by combining equations (7) and (8):

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{\partial J}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial \mathbf{w}}, \quad \frac{\partial J}{\partial b} = \frac{\partial J}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial b} \quad (9)$$

Differentiating equation (3) with respect to a ,

$$da = \frac{\partial J}{\partial a} = \frac{1-y}{1-a} - \frac{y}{a} \quad (10)$$

Differentiating equation (2) with respect to z ,

$$\frac{\partial a}{\partial z} = \frac{-1}{(1+e^{-z})^2} \cdot -e^{-z} = \frac{e^{-z}}{(1+e^{-z})^2} \quad (11)$$

Using the relation $a = \frac{1}{1+e^{-z}}$, we can write:

$$e^{-z} = \frac{1}{a} - 1 \quad (12)$$

Using equations (11) and (12), we can write:

$$\frac{\partial a}{\partial z} = a(1-a) \quad (13)$$

Using equations (13) and (10):

$$dz = \frac{\partial J}{\partial z} = \left(\frac{1-y}{1-a} - \frac{y}{a} \right) \cdot a(1-a) = a-y \quad (14)$$

Additionally, using equation (1) and the fact that $\mathbf{w} = [w_1, w_2, \dots, w_n]$:

$$\frac{\partial z}{\partial w_i} = x_i, \quad \frac{\partial z}{\partial b} = 1$$

Using the above relations and relation (9),

$$dw_i = \frac{\partial J}{\partial w_i} = (a-y) \cdot x_i, \quad db = \frac{\partial J}{\partial b} = a-y \quad (15)$$

Now, these formulae can be used in the iterative process as defined in the Algorithm Structure in Section 3.2.

3.4 Logistic Regression With Multiple Training Examples

3.4.1 Vectorization

The above model of logistic regression was simplified to process a single input with n features. Now, this model can be generalized using matrices to effectively process m training examples at the same time. This enables the algorithm to perform parallel computations instead of looping over each example one at a time, thus making it more efficient. With n features and m training examples, we can transform $n \times 1$ vector $\mathbf{x}^{(i)}$ (where i represents the i^{th} training example) into an $n \times m$ matrix $\mathbf{X}$, where each column represents a different training example with n features. This can be represented as:

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \dots & \mathbf{x}^{(m)} \\ | & | & & | \end{bmatrix} = X = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(m)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \dots & x_n^{(m)} \end{bmatrix} \quad (16)$$

To use matrix $\mathbf{X}$ in equation (1), the scalar z of each training example is collected into a $1 \times m$ row vector $\mathbf{Z}$, and the scalar bias b is added to every one of its entries. This automatic copying of a single value across an entire vector is called “broadcasting”, and we will rely on it repeatedly. Thus, equation (1) can be rewritten as:

$$\mathbf{Z} = \mathbf{w}^T \mathbf{X} + b \quad (17)$$

Note that b stays a plain scalar here: with only one neuron there is a single bias value, simply broadcast across all m columns of $\mathbf{Z}$. This will change once a layer holds more than one neuron, since each neuron then carries its own bias, and b will become a column vector with one entry per neuron; that vector is in turn broadcast into an effective matrix across the training examples. Throughout, b is written in lowercase (bolded once it becomes a vector) rather than capitalized like $\mathbf{W}$, since it is only ever indexed by neuron and never gains the second, per-input dimension that forces $\mathbf{W}$ to become a full matrix. Subsequently, a would also turn into a $1 \times m$ row vector $\mathbf{A}$, obtained by applying the sigmoid to each entry of $\mathbf{Z}$ independently:

$$\mathbf{A} = \sigma(\mathbf{Z}) \quad (18)$$

where $\mathbf{A} = [a^{(1)} \ a^{(2)} \ \dots \ a^{(m)}]$, and we likewise collect the labels into $\mathbf{Y} = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}]$. This would cause the equation for the cost function J to be modified such that it can estimate the average loss in the predictions that it makes.

$$J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log (1 - a^{(i)})] \quad (19)$$

A subtlety appears the moment we vectorize the backward pass. The sigmoid in $\mathbf{A} = \sigma(\mathbf{Z})$ is applied to each entry separately: the activation $a^{(i)}$ depends on its own $z^{(i)}$ and on no other $z^{(j)}$. As a result, every cross derivative $\frac{\partial a^{(i)}}{\partial z^{(j)}}$ with $i \neq j$ is zero, and the

only derivatives that survive are the matching ones, $\frac{\partial a^{(i)}}{\partial z^{(i)}} = \sigma'(z^{(i)})$. When the chain rule carries a gradient from $\mathbf{A}$ back to $\mathbf{Z}$, it would in general sum, for each $z^{(i)}$, the contributions arriving through every $a^{(j)}$; but here all of those cross contributions are zero, so each entry of the incoming gradient is simply scaled by the derivative at that same entry. This entry-by-entry multiplication of two equally shaped vectors (or matrices) is the “elementwise”, or “Hadamard”, product, written with the symbol $\odot$ and defined by

$$(\mathbf{u} \odot \mathbf{v})_i = u_i v_i.$$

This is why the elementwise appears as it is the exact form the chain rule takes whenever the function in between acts on each entry independently. Because there are no cross terms left to sum, the usual matrix multiplication reduces to a scaling of matched entries. Writing this out for our case,

$$d\mathbf{Z} = d\mathbf{A} \odot \sigma'(\mathbf{Z}),$$

and substituting the single-example results $\sigma'(z) = a(1 - a)$ and $da = \frac{1-y}{1-a} - \frac{y}{a}$ from Section 3.3, the elementwise product collapses neatly to

$$d\mathbf{Z} = \mathbf{A} - \mathbf{Y}.$$

For the sigmoid paired with the cross-entropy loss this cancellation is exact, so the $\odot$ never has to be written out explicitly here; but the very same reasoning, without such a clean cancellation, is exactly what we will need for the hidden layer in Section 4. The gradients of the cost then follow as

$$d\mathbf{w} = \frac{1}{m} \mathbf{X} d\mathbf{Z}^T = \frac{1}{m} \mathbf{X}(\mathbf{A} - \mathbf{Y})^T, \quad db = \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}).$$

The update steps for the parameters $\mathbf{w}$ and b then stay the same as before, since their structure is preserved (note that the scalar bias b receives a single averaged gradient over all m examples):

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}} = \mathbf{w} - \alpha \cdot d\mathbf{w} \tag{20}$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b} = b - \alpha \cdot db \tag{21}$$

3.4.2 Algorithm Structure for Multiple Training Examples

Thus, using equations (17), (18), (19), (20), and (21) for p iterations over all m training examples, the algorithm can be represented as:

Repeat, for $k = 1$ to p : {

$$\mathbf{Z} = \mathbf{w}^T \mathbf{X} + b$$

$$\mathbf{A} = \sigma(\mathbf{Z})$$

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log (1 - a^{(i)})]$$

$$d\mathbf{Z} = \mathbf{A} - \mathbf{Y}$$

$$d\mathbf{w} = \frac{1}{m} \mathbf{X} d\mathbf{Z}^T = \frac{1}{m} \mathbf{X} (\mathbf{A} - \mathbf{Y})^T$$

$$db = \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)})$$

$$\mathbf{w} := \mathbf{w} - \alpha \cdot d\mathbf{w}$$

$$b := b - \alpha \cdot db$$

}

The difference in this implementation compared to that with a single training example is that with multiple training examples, the cost functions, along with the gradients, need to be averaged over the number of training examples taken. This ensures that each individual example contributes to the gradient and the scale of the gradient is also maintained.

4 Neural Network Structure

4.1 Shallow Neural Network

4.1.1 Architecture

A neural network has multiple layers with many "neurons" or nodes inside each layer. Each of these neurons essentially performs logistic regression on its own. A fairly simple structure for a neural network would be one with one hidden layer and one output layer. The structure for this can be built upon by the algorithm that we devised in Section 3.4.2 for multiple training examples.

Let the number of layers in the neural network be represented by L , and let the components of each layer be denoted by the superscript $^{[l]}$, where l is the layer number. The process for this neural network can be written as:

$$x \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]}$$

Let us look at the hidden layer. Let it have $n^{[1]}$ neurons. Thus, these $n^{[1]}$ neurons will all be performing logistic regression, which will be fed into the output layer as $n^{[1]}$ inputs. Since there are $n^{[1]}$ neurons, each neuron must have n parameters $(w_1, w_2, \dots, w_n)$, one for every input feature. As in Section 3.1, each neuron's weight vector $\mathbf{w}_i^{[1]}$ is stored as an $n \times 1$ column, matching $\mathbf{w}$ in the logistic-regression case; stacking the transposes of

the $n^{[1]}$ neurons of the layer as rows gives an $n^{[1]} \times n$ matrix for the whole layer.

$$\mathbf{W}^{[1]} = \begin{bmatrix} \text{---} & \mathbf{w}_1^{[1]T} & \text{---} \\ \text{---} & \mathbf{w}_2^{[1]T} & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{w}_{n^{[1]}}^{[1]T} & \text{---} \end{bmatrix} = \begin{bmatrix} w_{1,1}^{[1]} & w_{1,2}^{[1]} & \cdots & w_{1,n}^{[1]} \\ w_{2,1}^{[1]} & w_{2,2}^{[1]} & \cdots & w_{2,n}^{[1]} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n^{[1],1}}^{[1]} & w_{n^{[1],2}}^{[1]} & \cdots & w_{n^{[1],n}}^{[1]} \end{bmatrix}$$

Now, if the matrix is substituted in equation (17), this would cause the $1 \times m$ vector to transform into a $n^{[1]} \times m$ matrix $\mathbf{Z}^{[1]}$. As anticipated in Section 3.4.1, since this layer holds $n^{[1]}$ neurons, the bias becomes a column vector $\mathbf{b}^{[1]}$ of shape $n^{[1]} \times 1$, holding one bias per neuron, which is broadcast across all m columns (so that every training example in a given row is offset by the same bias). Thus, the equation can be rewritten as:

$$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]}\mathbf{X} + \mathbf{b}^{[1]} \quad (22)$$

Subsequently, the row vector $\mathbf{A}$ will also be transformed to the matrix $\mathbf{A}^{[1]}$, with the same dimensions as $\mathbf{Z}^{[1]}$, by applying an activation function $g^{[1]}(\cdot)$ to it entry by entry. At this point the activation must be chosen, and here we make a deliberate departure from logistic regression: we do not reuse the sigmoid for the hidden layer. The sigmoid saturates, meaning that for inputs z that are even moderately large in magnitude its curve flattens out and its derivative $a(1-a)$ becomes vanishingly small. When many such small derivatives are multiplied together through the chain rule during backpropagation, the gradients reaching the earlier layers shrink towards zero and those layers effectively stop learning; this is the well-known “vanishing gradient” problem. To avoid it, we use the “Rectified Linear Unit” (ReLU) (Nair and Hinton) for the hidden layer, defined as

$$g^{[1]}(z) = \text{ReLU}(z) = \max(0, z).$$

ReLU is trivial to compute, and for every positive input its derivative is exactly 1, so gradients flow back through the hidden layer undiminished and training stays fast. Its derivative is simply

$$g^{[1]'}(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}.$$

The hidden-layer activation is therefore

$$\mathbf{A}^{[1]} = g^{[1]}(\mathbf{Z}^{[1]}) = \text{ReLU}(\mathbf{Z}^{[1]}) \quad (23)$$

Now, $\mathbf{A}^{[1]}$ will be fed into the output layer (2^{nd} layer) as input. Since the output layer has only 1 neuron, the matrix $\mathbf{W}^{[2]}$ will actually be a $1 \times n^{[1]}$ vector, and the bias $b^{[2]}$ is a single scalar broadcast across the m columns.

$$\mathbf{Z}^{[2]} = \mathbf{W}^{[2]}\mathbf{A}^{[1]} + b^{[2]} \quad (24)$$

Subsequently, $\mathbf{Z}^{[2]}$ and $\mathbf{A}^{[2]}$ will take a shape of $1 \times m$. For the output layer, however, we do keep the sigmoid: unlike a hidden unit, the final neuron has to produce a number in

$(0, 1)$ that we can read as the probability $P(y = 1 \mid \mathbf{x})$, and the sigmoid is precisely the function that guarantees this. Writing the output activation as $g^{[2]}(z) = \sigma(z)$,

$$\mathbf{A}^{[2]} = g^{[2]}(\mathbf{Z}^{[2]}) = \sigma(\mathbf{Z}^{[2]}) \quad (25)$$

This division of labour, ReLU in the hidden layer to keep gradients healthy and sigmoid at the output to produce a probability, is the standard choice for a binary classifier.

4.1.2 Backpropagation

In this shallow neural network, since there are two layers, we would need to find the derivatives of the parameters of both layers and update them. To do so, we can use computation graphs and backpropagate to find the gradients $d\mathbf{W}^{[1]}$, $d\mathbf{b}^{[1]}$, $d\mathbf{W}^{[2]}$ and $d\mathbf{b}^{[2]}$. The structure of the graph is as follows:

$$\mathbf{W}^{[1]} \rightarrow \mathbf{Z}^{[1]} \rightarrow \mathbf{A}^{[1]} \rightarrow \mathbf{W}^{[2]} \rightarrow \mathbf{Z}^{[2]} \rightarrow \mathbf{A}^{[2]} \rightarrow J$$

The process for finding $d\mathbf{W}^{[2]}$ and $db^{[2]}$ is identical to what we have performed before in the process leading up to equation (15), and the equations just need to be vectorized. Exactly as in the logistic-regression output, the gradient entering the linear step of the output layer is $d\mathbf{Z}^{[2]} = \mathbf{A}^{[2]} - \mathbf{Y}$. Thus, we can write:

$$d\mathbf{W}^{[2]} = \frac{1}{m} d\mathbf{Z}^{[2]} \mathbf{A}^{[1]T} = \frac{1}{m} (\mathbf{A}^{[2]} - \mathbf{Y}) \mathbf{A}^{[1]T}, \quad db^{[2]} = \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) \quad (26)$$

Now, we can backpropagate the derivatives using the chain rule to find $d\mathbf{W}^{[1]}$ and $d\mathbf{b}^{[1]}$.

$$\frac{\partial J}{\partial \mathbf{A}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \times \frac{\partial \mathbf{Z}^{[2]}}{\partial \mathbf{A}^{[1]}} \quad (27)$$

$$\frac{\partial J}{\partial \mathbf{Z}^{[1]}} = \frac{\partial J}{\partial \mathbf{A}^{[1]}} \times \frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} \quad (28)$$

$$\frac{\partial J}{\partial \mathbf{W}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[1]}} \times \frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{W}^{[1]}}, \quad \frac{\partial J}{\partial \mathbf{b}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[1]}} \times \frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{b}^{[1]}} \quad (29)$$

Combining equations (27), (28) and (29),

$$\frac{\partial J}{\partial \mathbf{W}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \times \frac{\partial \mathbf{Z}^{[2]}}{\partial \mathbf{A}^{[1]}} \times \frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} \times \frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{W}^{[1]}} \quad (30)$$

$$\frac{\partial J}{\partial \mathbf{b}^{[1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[2]}} \times \frac{\partial \mathbf{Z}^{[2]}}{\partial \mathbf{A}^{[1]}} \times \frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} \times \frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{b}^{[1]}} \quad (31)$$

We know that:

$$d\mathbf{Z}^{[2]} = \mathbf{A}^{[2]} - \mathbf{Y}$$

Using this in equation (27),

$$\frac{\partial J}{\partial \mathbf{A}^{[1]}} = \left(\frac{\partial \mathbf{Z}^{[2]}}{\partial \mathbf{A}^{[1]}} \right)^T d\mathbf{Z}^{[2]} = \mathbf{W}^{[2]T} d\mathbf{Z}^{[2]}$$

We can also infer that:

$$\frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} = g^{[1]'}(\mathbf{Z}^{[1]})$$

Substituting this in equation (28),

$$d\mathbf{Z}^{[1]} = \frac{\partial J}{\partial \mathbf{A}^{[1]}} \odot \frac{\partial \mathbf{A}^{[1]}}{\partial \mathbf{Z}^{[1]}} = \mathbf{W}^{[2]T} d\mathbf{Z}^{[2]} \odot g^{[1]'}(\mathbf{Z}^{[1]})$$

This expression shows precisely why one factor is combined as a matrix product and the other as the elementwise product introduced in Section 3.4.1. The first factor, $d\mathbf{A}^{[1]} = \mathbf{W}^{[2]T} d\mathbf{Z}^{[2]}$, comes from the relation $\mathbf{Z}^{[2]} = \mathbf{W}^{[2]}\mathbf{A}^{[1]} + b^{[2]}$. Here each entry of $\mathbf{Z}^{[2]}$ is a weighted sum of every entry of $\mathbf{A}^{[1]}$, so the derivative of $\mathbf{Z}^{[2]}$ with respect to $\mathbf{A}^{[1]}$ has non-zero cross terms, and the chain rule at this step is a full matrix product. The second factor comes from $\mathbf{A}^{[1]} = g^{[1]}(\mathbf{Z}^{[1]})$, where the ReLU acts on each entry of $\mathbf{Z}^{[1]}$ separately. Using the argument in Section 3.4.1, its cross derivatives are zero, so this step reduces to the elementwise product with the derivative $g^{[1]'}(\mathbf{Z}^{[1]})$. In short, the weight matrix connects every input to every output and therefore requires a matrix product, whereas the activation connects each entry only to itself and therefore requires an elementwise product.

For the ReLU, that derivative equals 1 wherever the corresponding entry of $\mathbf{Z}^{[1]}$ was positive and 0 otherwise, so the elementwise product keeps the gradient for the units that were active on the forward pass and removes it for the rest. Unlike the output layer, where the sigmoid and the cross-entropy loss cancelled to leave the clean $\mathbf{A}^{[2]} - \mathbf{Y}$, no such cancellation occurs here, so the elementwise product stays in the final expression. Substituting this in equation (29),

$$d\mathbf{W}^{[1]} = \frac{1}{m} d\mathbf{Z}^{[1]} \mathbf{X}^T = \frac{1}{m} \left(\mathbf{W}^{[2]T} d\mathbf{Z}^{[2]} \odot g^{[1]'}(\mathbf{Z}^{[1]}) \right) \mathbf{X}^T \quad (32)$$

The bias gradient follows the same substitution into equation (29). Here $\frac{\partial \mathbf{Z}^{[1]}}{\partial \mathbf{b}^{[1]}} = 1$, exactly as it was for b in Section 3.3, so $d\mathbf{b}^{[1]}$ is just $d\mathbf{Z}^{[1]}$ itself, averaged across the m training examples. Since $\mathbf{b}^{[1]}$ has one entry per neuron rather than a single scalar, this average is taken across the columns of $d\mathbf{Z}^{[1]}$ only, collapsing the m examples while keeping the $n^{[1]}$ neurons separate:

$$d\mathbf{b}^{[1]} = \frac{1}{m} \sum_{i=1}^m d\mathbf{Z}^{[1](i)} \quad (33)$$

where $d\mathbf{Z}^{[1](i)}$ denotes the i^{th} column of $d\mathbf{Z}^{[1]}$, so that $d\mathbf{b}^{[1]}$ comes out with the same $n^{[1]} \times 1$ shape as $\mathbf{b}^{[1]}$ itself.

With $d\mathbf{W}^{[1]}$, $d\mathbf{b}^{[1]}$, $d\mathbf{W}^{[2]}$, and $d\mathbf{b}^{[2]}$ all computed, both layers are updated using the same gradient descent rule introduced in Section 3.1, applied once per layer:

$$\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha \cdot d\mathbf{W}^{[l]}, \quad \mathbf{b}^{[l]} := \mathbf{b}^{[l]} - \alpha \cdot d\mathbf{b}^{[l]}, \quad l = 1, 2.$$

4.1.3 Algorithm

Now, we have all the components necessary to start building the algorithm for this shallow neural network. Repeat, for $k = 1$ to p : {

$$\begin{aligned}
\mathbf{Z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{X} + \mathbf{b}^{[1]} \\
\mathbf{A}^{[1]} &= \text{ReLU}(\mathbf{Z}^{[1]}) \\
\mathbf{Z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{A}^{[1]} + b^{[2]} \\
\mathbf{A}^{[2]} &= \sigma(\mathbf{Z}^{[2]}) \\
J(\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, b^{[2]}) &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log (1 - a^{(i)})] \\
d\mathbf{Z}^{[2]} &= \mathbf{A}^{[2]} - \mathbf{Y} \\
d\mathbf{W}^{[2]} &= \frac{1}{m} d\mathbf{Z}^{[2]} \mathbf{A}^{[1]T} \\
db^{[2]} &= \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) \\
d\mathbf{Z}^{[1]} &= \mathbf{W}^{[2]T} d\mathbf{Z}^{[2]} \odot g^{[1]'}(\mathbf{Z}^{[1]}) \\
d\mathbf{W}^{[1]} &= \frac{1}{m} d\mathbf{Z}^{[1]} \mathbf{X}^T \\
d\mathbf{b}^{[1]} &= \frac{1}{m} \sum_{i=1}^m d\mathbf{Z}^{[1](i)} \\
\mathbf{W}^{[1]} &:= \mathbf{W}^{[1]} - \alpha \cdot d\mathbf{W}^{[1]} \\
\mathbf{b}^{[1]} &:= \mathbf{b}^{[1]} - \alpha \cdot d\mathbf{b}^{[1]} \\
\mathbf{W}^{[2]} &:= \mathbf{W}^{[2]} - \alpha \cdot d\mathbf{W}^{[2]} \\
b^{[2]} &:= b^{[2]} - \alpha \cdot db^{[2]} \\
\}
\end{aligned}$$

Unlike the earlier algorithms, this one carries two full sets of parameters, $(\mathbf{W}^{[1]}, \mathbf{b}^{[1]})$ and $(\mathbf{W}^{[2]}, b^{[2]})$, that are updated together on every iteration. The same pattern, computing each layer's linear step, applying an activation, and then working backwards one layer at a time to accumulate gradients, extends unchanged to networks with more than two layers; only the number of times it is repeated grows, not the underlying idea.

4.2 Deep Neural Networks

We can now generalize the process repeated during the construction of the shallow neural network to form deep neural networks for binary classification. Let the number of layers in the network be L , and let the number of nodes in each layer be $n^{[l]}$, where l represents the layer number. Additionally, we can write $\mathbf{X}$ as $\mathbf{A}^{[0]}$ to be able to generalize the equations more effectively.

Thus, for the forward propagation of the network up to the second-to-last layer, we can write:

Repeat, for $l = 1$ to $L - 1$: {

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

}

For the output layer,

$$\mathbf{Z}^{[L]} = \mathbf{W}^{[L]} \mathbf{A}^{[L-1]} + b^{[L]}$$

$$\mathbf{A}^{[L]} = g^{[L]}(\mathbf{Z}^{[L]})$$

As in Section 4.1.1, $g^{[l]}$ for $l < L$ is typically the ReLU, while $g^{[L]}$ is the sigmoid, since the network still ends in a binary classification decision regardless of how many hidden layers precede it.

This shows that the forward propagation steps for the neural network are fairly easy to devise and implement. The backward propagation steps pose a bigger challenge. For the backward propagation, we start at the output layer L and initialize, for each training example i ,

$$d\mathbf{A}^{[L](i)} = \frac{1}{m} \left[\frac{1 - y^{(i)}}{1 - a^{[L](i)}} - \frac{y^{(i)}}{a^{[L](i)}} \right], \quad i = 1, \dots, m,$$

which is simply the single-example result $da = \frac{1-y}{1-a} - \frac{y}{a}$ from equation (10), computed independently in each column and scaled by $\frac{1}{m}$ to match the averaged cost function.

Thereafter, we start the process of computing derivatives backwards for each layer. Since each layer's backward step depends on $d\mathbf{A}^{[l]}$ handed down from the layer after it, this loop must run in the opposite direction to the forward pass, starting at the output layer and working back towards the input:

Repeat, for $l = L$ down to 1: {

$$d\mathbf{Z}^{[l]} = d\mathbf{A}^{[l]} \odot g^{[l]'}(\mathbf{Z}^{[l]})$$

$$d\mathbf{W}^{[l]} = \frac{1}{m} d\mathbf{Z}^{[l]} \mathbf{A}^{[l-1]T}$$

$$d\mathbf{b}^{[l]} = \frac{1}{m} \sum_{i=1}^m d\mathbf{Z}^{[l](i)}$$

$$d\mathbf{A}^{[l-1]} = \mathbf{W}^{[l]T} d\mathbf{Z}^{[l]}$$

}

5 Implementation

5.1 Familiarization With the Dataset

The Higgs Boson dataset has 200,000 rows with 30 features of events, along with a binary label of whether an event was a Higgs Boson signal event (1) or background noise (0).

At the LHC (Large Hadron Collider), protons are smashed together at extremely high energy, producing sprays of particles that detectors measure. Most of these collisions produce ordinary, well-understood physics processes ("background"). Very rarely, a collision produces a Higgs boson, which almost immediately decays into other particles before it can be directly observed. The pattern of particles that come out afterward (their energies, momenta, angles) could indicate whether the particles come from a Higgs decay versus other common physics processes.

The split of signal events to background noise in the dataset is $\sim 0.657 : 0.343$, i.e. always predicting "background" would yield an accuracy of 65.7%.

5.2 Pre-processing the Data

Before the data can be fed into any model, it first needs to be pre-processed to ensure that it is in the correct format for the required input. To do this, some features of the data, such as the "EventD" and "Weight", were removed from the data as they would not have had any impact while training. The dataset did not contain some values of "-999", which were used to denote "NA". Therefore, those values were also filtered out during the preprocessing. Additionally, the data was converted from a .csv format to a NumPy array to aid in computation.

Further, the data was also normalized. This was done for a few reasons. Firstly, since different features are spread between different orders of magnitude, those that are large in raw value would dominate the $\mathbf{w}^T \mathbf{x} + b$ computation. Thus, the magnitude of the weight given to a feature would represent how it impacts the final outcome rather than how big or small the feature is. This also prevents saturation in activation functions, especially in the sigmoid activation, as extreme values would push the function to either 0 or 1, which has a very low rate of change, and there are greater chances for the weights to be "stuck" in such low-gradient areas.

An 80 : 20 split (200,000 : 50,000) was used to separate the 250,000 rows of data into training and testing sets, according to common convention.

5.3 Initialization of Parameters

Since initializing the weight matrices to zero would essentially mean that every neuron in the layer would give an identical result, the values of the weight matrices of each layer were initialized to random values using the `np.random.randn()` function and were scaled by a factor of $\sqrt{\frac{2}{l}}$, where l is the number of neurons in the previous layer of the neural network. This is called the "He" initialization, introduced by Kaiming He and his colleagues in 2015 specifically to work with ReLU activations (He et al.). This type of scaling helps prevent vanishing or exploding gradients.

5.4 Code Structure

To implement my ideas for the neural networks in code, I identified a set of 4 major steps that have to be performed during every iteration:

1. Forward propagation
2. Computing the cost for the iteration
3. Backward propagation
4. Updating parameters

I created a set of 4 main functions to implement each of these steps: `model_forward()`, `compute_cost()`, `model_backward()`, and `update_parameters()`. Additionally, I created a function, `run_model()`, to call all of the above functions cohesively for each iteration of training, along with a function, `predict()`, to use the trained neural network to make predictions and check the accuracy on a given set.

For both the forward and backward passes, I implemented a 3-tier function structure to compute them:

- The bottom-most tier, `linear_forward()` and `linear_backward()`, computes the linear part of each pass, i.e. $\mathbf{Z}^{[l]} = \mathbf{W}^{[l]}\mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$ and its corresponding gradients, given the layer's parameters.
- The second tier, `linear_activation_forward()` and `linear_activation_backward()`, wraps the bottom tier and additionally applies the correct activation function, since different activations require different methods to compute their forward and backward passes.
- The top tier, `model_forward()` and `model_backward()`, calls the second tier layer by layer across the entire network, selecting the appropriate activation function (ReLU for hidden layers, sigmoid for the output layer) at each step.

Additionally, I defined several helper functions to compute the activation functions and their derivatives directly: `sigmoid()`, `relu()`, `sigmoid_backward()`, and `relu_backward()`.

The whole codebase for the functions is available on my GitHub repository, as well as the appendix of this report.

5.5 Testing

5.5.1 Logistic Regression on a Single Neuron

To put the algorithm to the test, a logistic regression model was implemented, on which the data was trained. This model showed an accuracy of $\sim 75.0\%$ on the training set and $\sim 74.6\%$ accuracy on the test set at a learning rate of $r = 0.01$ for 10,000 iterations, which is already better than the baseline accuracy of 65.7%, indicating that a refined model could possibly yield better predictions. With these results, I decided to implement a neural network to understand how far accurate the predictions could be.

5.6 Deep Neural Network

Looking at the improvements from the single neuron model, I decided to implement a deep neural network to test its accuracy. It was composed of 5 layers, and the following architecture of neurons in each layer:

$$\mathbf{A}^{[0]} \rightarrow 20 \rightarrow 10 \rightarrow 7 \rightarrow 4 \rightarrow 1$$

5.7 Hyperparameter Tuning

5.7.1 Learning Rate

Initially, when I tried training it at a learning rate of $r = 0.01$, I found the accuracy to be $\sim 78\%$ for both the training and the test sets. Then, looking at the cost curve, I realized that the model was far away from convergence. Thus, I decided to train the model on a number of learning rates:

$$[0.01, 0.025, 0.075, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]$$

The cost curves of the models and their accuracies can be seen in the following figures.

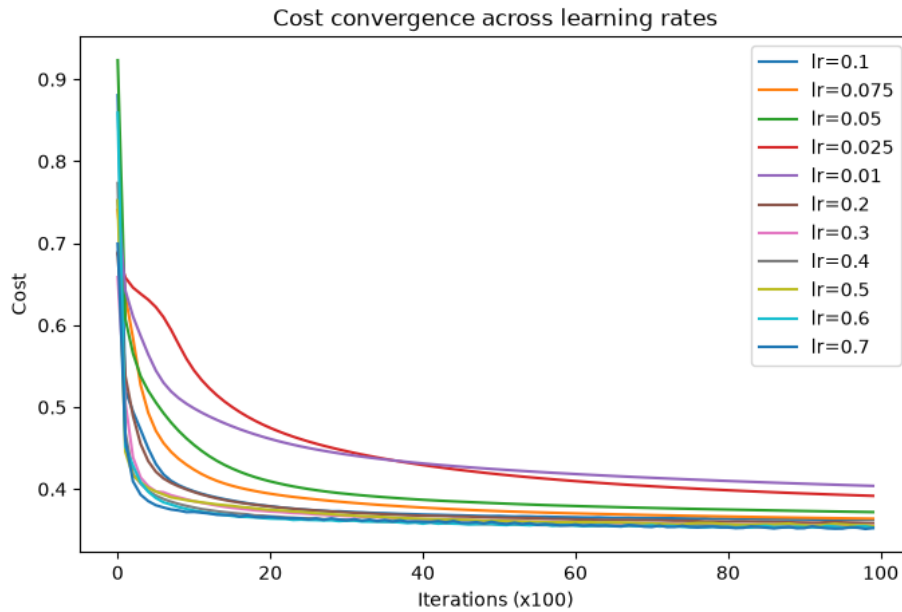


Figure 1: Cost curve for all the different learning rates tested

Figure 1 shows that a higher learning rate converges faster and generally at a lower cost for a fixed number of iterations. We do have to note that this cannot be generalized as once a learning rate gets too big, it can fail to find the optima as it could take steps that would be too big as well. When that is the case, the graph for cost would not show a

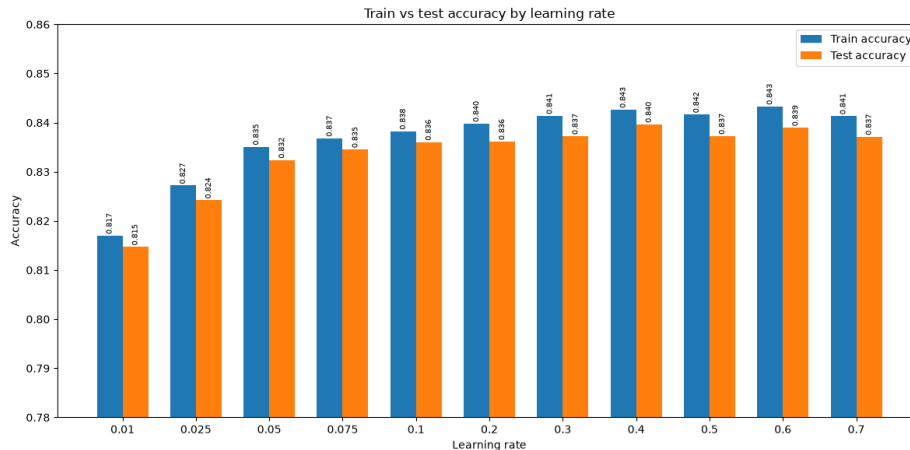


Figure 2: Graph of the train and test accuracies for each learning rate

smooth curve but one that has many spikes in between, and early signs of these spikings can be seen if one looks closely at the plot for $r = 0.7$.

Looking at Figure 2, the model with a learning rate of $r = 0.6$ showed the highest training accuracy of $\sim 84.3\%$, along with a test accuracy of $\sim 83.9\%$. This echoes the findings from Figure 1, as we see that a higher learning rate has enabled the model to reach higher accuracy within a fixed number of iterations but the highest learning rate is not necessarily the best one to use.

This showed me that finding the best neural network model for a specific dataset and use case is not extremely obvious, and requires quite a lot of trial and error.

5.7.2 Model Architecture

Since experimenting with the learning rate yielded a variety of results, I thought I could experiment with different model architectures as well to explore how it can affect and help improve accuracy.

There were some changes that were made to the implementation in code with this decision. Previously, training these neural networks for 11 learning rates was taking a very long time (around 5 hours) on my laptop. Examining this, I knew there had to be a method to train faster as I had seen frameworks like TensorFlow train such models with a similar dataset size in a much shorter time. While researching, I came across a python package called `cupy` (CuPy Developers), which has all the same functions as `numpy`, along with the ability to compute matrix multiplications using the GPU. This also needed no changes to the codebase of my neural network, as I could simply swap packages while preserving all functions and code I had written previously. Thus, I was able to easily implement this change and train on my modest laptop GPU.

Additionally, I also came across the idea of training the network in mini-batches while researching. In this method, the data is divided into a number of small batches, and the model processes each batch alone and updates parameters after the pass is run for each batch. An epoch is completed after each batch is processed, and the model is trained for a number of epochs similar to the number of iterations we used earlier). This is essentially the process frameworks like TensorFlow and PyTorch use (TensorFlow Developers), which enables them to train very large networks in a relatively short amount of time. This is computationally cheaper, as the model does not have to perform a full pass with the whole 200,000 rows of the dataset. Instead, it gets the opportunity to update its parameters more frequently, which in many cases results in a faster path to convergence. Additionally, since it does not process the full size of the dataset at once, it is more memory efficient and lets models train on a small amount of available memory as well.

Integrating both of these changes helped me train 8 different model architectures that were more complex in a much shorter amount of time (~ 75 minutes). The architectures are seen in the following table.

Architecture	Hidden layer widths	Total layers	Parameters
Baseline (previous)	20, 10, 7, 4	5	944
Double width	40, 20, 14, 8	5	2,483
Extra layer	40, 20, 10, 7, 4	6	2,384
Two extra layers	80, 40, 20, 10, 7, 4	7	6,864
Wide	128, 64	3	12,289
Very wide	256, 128	3	40,961
Deep and wide	128, 64, 32, 16, 8	6	14,977
Oversized	512, 256, 128, 64	5	188,417

Table 1: Architectures tested, given by the number of neurons in each hidden layer, along with total parameter count. Parameter count for layer l is $n^{[l-1]} \times n^{[l]} + n^{[l]}$ (weights plus biases), summed across all layers. All networks share the same 30-feature input layer and single-neuron sigmoid output layer.

As I started training these models on the best learning rate I had before $r = 0.6$, I started getting `nan` in the cost computation I was logging, which was a signal that the cost had exploded. Upon researching, I realized that models with mini-batching implemented had to be trained on far lower learning rates, and complex models needed to have more batches to prevent the cost from exploding. Thus, I chose to train my models at a learning rate of $r = 0.01$, for 400 epochs by dividing the data into batches of 1024. The cost curves and accuracies of the models are shown in the following figures. An important point to note here is that I was not able to train my "oversized" model fully with this learning rate, as the cost exploded.

Upon analyzing the graphs, one can notice that the best result was obtained from the "deep and wide" model, with 6 layers and 14,977 parameters, which yielded a training accuracy of 85.4% but did not score too well in terms of the testing accuracy (83.4%). There is a clear observation I could make from this experiment. As the model architecture gets deeper and wider, the training accuracy continues to grow in minor increments, while the testing accuracy stays at a plateau. This is also evidenced by the cost curve of all of

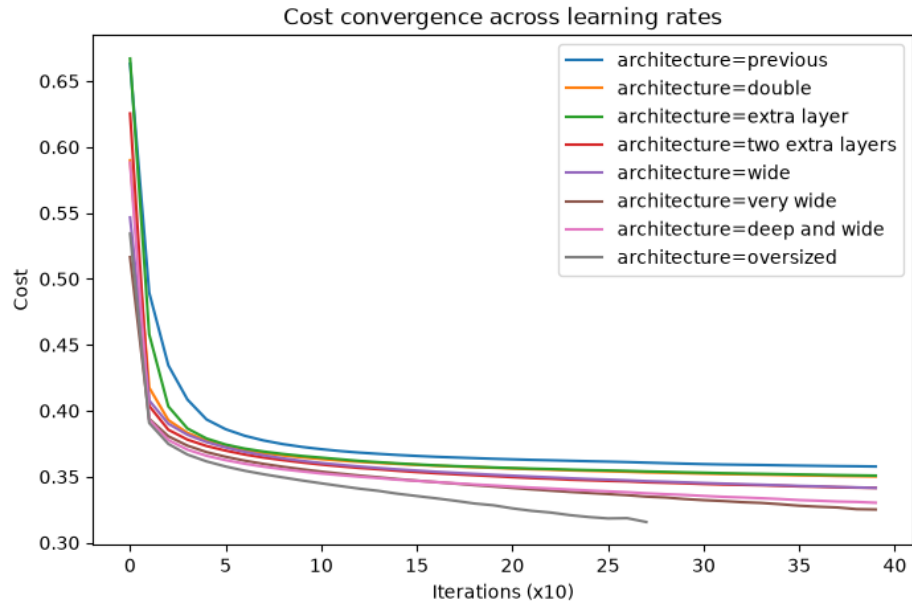


Figure 3: Cost curve for all the different model architectures tested

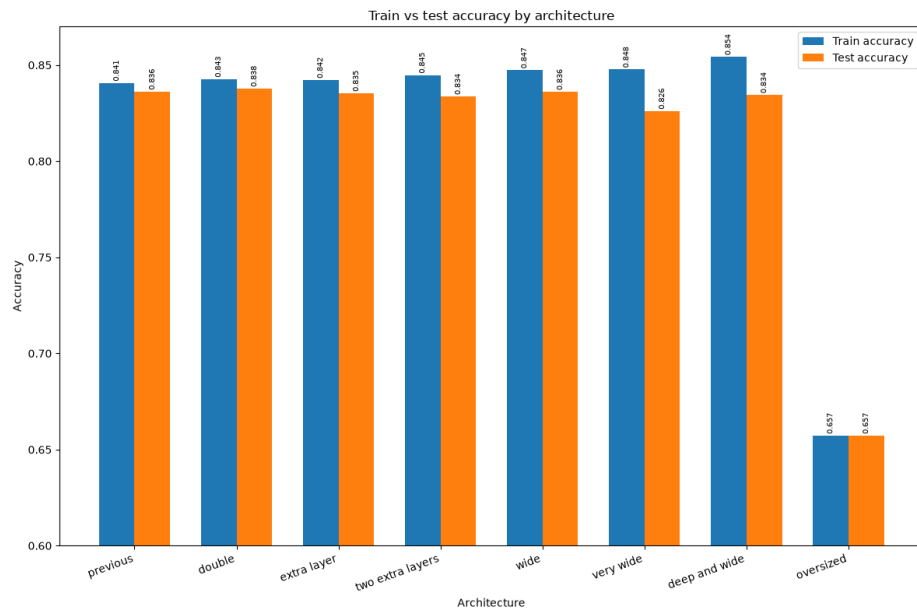


Figure 4: Train and test accuracies of the different model architectures tested

the models, especially the “oversized” one, which could reach to much lower costs while training before the cost exploded. Thus, it can be concluded that increasing the number of parameters has an effect on minimizing the cost more efficiently.

5.8 Evaluation

Since the dataset was used for the ATLAS Challenge competition, there have been many attempts at training different types of machine learning algorithms to try to achieve the best accuracy at prediction. I found a paper by S. Raza Ahmad which similarly implemented a deep neural network from scratch using NumPy for this same competition, achieving an accuracy of 83% using a 4-hidden-layer network with 300 units per layer, sigmoid activations throughout, and layer-wise pretraining using stacked autoencoders, trained across a 12-node distributed cluster over 7-10 days (Ahmad). It is worth noting that this comparison is not perfectly like-for-like given the substantial differences in architecture, compute, and training methodology; nonetheless, my comparatively lightweight, single-model network achieved a marginally higher accuracy of $\sim 84\%$.

This suggests that, for this dataset, a comparatively small, single from-scratch network trained on a laptop GPU was able to match, or even marginally exceed, the accuracy of a much larger and more heavily engineered deep learning solution built with far greater compute and time. This does not mean my implementation is inherently better; the referenced work explored a broader set of optimization techniques (momentum, RMSProp, learning rate annealing, layer-wise pretraining) that were outside the scope of this project. Rather, it suggests that for this particular dataset and feature set, the primary bottleneck on achievable accuracy is likely the amount of learnable signal present in the 30 features themselves, rather than model scale or training sophistication.

6 Conclusion

Thus, from this project, I have learned how to implement and fine-tune a neural network from scratch. Understanding the underlying mathematical mechanics of neural networks helped me understand how to construct and structure one, from making a choice of the activation functions to implementing it all in code. This gave me a much deeper intuition about how neural networks work as opposed to stacking some layers with a package like TensorFlow.

Deriving backpropagation by hand, from the single-neuron case in to the fully vectorized version, helped me understand concepts that previously felt like black-box function calls. Additionally, testing the model empirically taught me just as much as the derivations did. Comparing learning rates and model architecture showed me that convergence speed and the quality of the final solution are two different problems that might not be solved together.

Going forward, I plan to apply this same from-scratch approach to convolutional neural networks, and learn about model assessment, evaluation, and regularization to build a full machine learning project.

7 Works Cited

- Ahmad, S. Raza. “Technical Report of Participation in Higgs Boson Machine Learning Challenge.” *arXiv*, 9 Oct. 2015, <https://arxiv.org/abs/1510.02674>.
- CuPy Developers. “CuPy: NumPy & SciPy for GPU.” *CuPy*, <https://cupy.dev>.
- He, Kaiming, et al. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.” *arXiv*, 6 Feb. 2015, <https://arxiv.org/abs/1502.01852>.
- “Higgs Boson Machine Learning Challenge.” *Kaggle*, 2014, <https://www.kaggle.com/c/higgs-boson>.
- Nair, Vinod, and Geoffrey E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines.” *Proceedings of the 27th International Conference on Machine Learning*, 2010, <https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf>.
- Ng, Andrew. “Neural Networks and Deep Learning.” *Coursera*, DeepLearning.AI, <https://www.coursera.org/learn/neural-networks-deep-learning>.
- TensorFlow Developers. “TensorFlow Core.” *TensorFlow*, <https://www.tensorflow.org>.
- Kelkar, Pratham. “Neural-Network-from-Scratch.” *GitHub*, 2026, <https://www.github.com/prathamkelkar/Neural-Network-from-Scratch>.

8 Appendix

Listing 1: Preprocessing the Higgs data using preprocess_data_higgs.py

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4
5
6 def preprocess_data():
7
8     data = pd.read_csv("training/training.csv")
9     print(data.shape)
10    print(data.columns.tolist())
11    print(data["Label"].value_counts(normalize=True))
12
13    # Convert label to binary
14    Y = (data["Label"] == "s").astype(int).values
15
16    # Drop non-feature columns
17    drop_cols = ["EventId", "Label", "Weight"]
18    feature_cols = [c for c in data.columns if c not in drop_cols]
19
20    X = data[feature_cols].values
21
22    X_clean = X.copy()
23    for col_idx in range(X.shape[1]):
24        col = X_clean[:, col_idx]
25        missing_mask = col == -999.0
26        if missing_mask.sum() > 0:
27            valid_median = np.median(col[~missing_mask])
28            col[missing_mask] = valid_median
29
30    X_train_raw, X_test_raw, Y_train_raw, Y_test_raw =
31        train_test_split(
32            X_clean, Y, test_size=0.2, random_state=42, stratify=Y
33        )
34
35    X_train = X_train_raw.T
36    X_test = X_test_raw.T
37    Y_train = Y_train_raw.reshape(1, -1)
38    Y_test = Y_test_raw.reshape(1, -1)
39
40    mean = X_train.mean(axis=1, keepdims=True)
41    std = X_train.std(axis=1, keepdims=True)
42
43    X_train = (X_train - mean) / std
44    X_test = (X_test - mean) / std
45
46    return X_train, Y_train, X_test, Y_test
```


Listing 2: Neural network functions - NN_helper_functions.py

```

1 import cupy as np
2 import copy
3
4 def sigmoid(Z):
5     """
6     A function to implement the sigmoid activation on a numpy
7     ndarray Z
8
9     Arguments:
10    Z -- a numpy array of any shape
11
12    Returns:
13    A -- output of the sigmoid activation on Z -> it has the same
14    shape as Z
15    cache -- returns Z in the form of cache for use during
16    backpropagation
17    """
18
19    A = 1 / (1 + np.exp(-Z))
20    cache = Z
21
22    return A, cache
23
24 def relu(Z):
25     """
26     A function to implement the ReLU activation on a numpy ndarray Z
27
28     Arguments:
29     Z -- output of the linear layer of a neuron
30
31     Returns:
32     A -- output of the ReLU activation on Z -> it has the same shape
33     as Z
34     cache -- returns Z in the form of cache for use during
35     backpropagation
36     """
37
38    A = np.maximum(0, Z)
39    cache = Z
40
41    return A, cache
42
43 def relu_backward(dA, cache):
44     """
45     A function to implement the backpropagation for a relu neuron.
46
47     Arguments:
48     dA -- post-activation gradient
49     cache -- 'Z' computed before, for implementation backpropagation

```

```

45
46 Returns:
47 dZ -- Gradient of the cost with respect to Z
48 """
49
50 Z = cache
51 # initializing dZ to a numpy array with values of dA as the ReLU
52   function is linear
53 dZ = np.array(dA, copy=True)
54
55 # Setting the parts of dZ as 0 where Z is also 0
56 dZ[Z <= 0] = 0
57
58 return dZ
59
60 def sigmoid_backward(dA, cache):
61     """
62     A function to implement the backpropagation for a sigmoid neuron
63     .
64
65     Arguments:
66     dA -- post-activation gradient
67     cache -- 'Z' computed before, for implementation backpropagation
68
69     Returns:
70     dZ -- Gradient of the cost with respect to Z
71     """
72
73     Z = cache
74
75     s = 1 / (1 + np.exp(-Z))
76     dZ = dA * s * (1-s)
77
78     return dZ
79
80 def initialize_parameters(layer_dims):
81     """
82     A function to initialize all the parameters (w and b numpy
83     arrays) before training.
84
85     Arguments:
86     layer_dims -- python list containing the number of neurons of
87     each layer in the neural network
88
89     Returns:
90     parameters -- python dictionary coontaining parameters "W1", "b1
91     ", "W2", "b2", ... "WL", "bL":
92         Wl -- matrix containing weights
93         bl -- bias vector

```

```

90     """
91
92     parameters = {}
93     # extracting number of layers in the neural network
94     L = len(layer_dims)
95
96     for l in range(1, L):
97         parameters['W' + str(l)] = np.random.randn(layer_dims[l],
98             layer_dims[l-1]) * np.sqrt(2 / layer_dims[l-1])
99         parameters['b' + str(l)] = np.zeros((layer_dims[l], 1))
100
101     return parameters
102
103 def linear_forward(A, W, b):
104     """
105     A function to apply the linear function to a layer during
106     forward propagation.
107
108     Arguments:
109     A -- activation matrix from previous layer
110     W -- weight matrix of the current layer
111     b -- bias vector of the current layer
112
113     Returns:
114     Z -- the input that goes into the activation function
115     cache -- tuple containing 'A', 'W', 'b', stored as cache for
116         backpropagation computation
117
118     """
119
120     Z = np.dot(W, A) + b
121
122     cache = (A, W, b)
123
124     return Z, cache
125
126 def linear_activation_forward(A_prev, W, b, activation):
127     """
128     A function to implement the forward propagation for a layer
129
130     Arguments:
131     A_prev -- activation matrix from previous layer
132     W -- weight matrix of current layer
133     b -- bias vector of current layer
134
135     Returns:
136     A -- the output of the activation function
137     cache -- tuple containing ((Z, ), (A, W, b)), stored as cache
138         for backpropagation computation
139
140     """

```

```

136
137     Z, linear_cache = linear_forward(A_prev, W, b)
138
139     if activation == "sigmoid":
140         A, activation_cache = sigmoid(Z)
141
142     elif activation == "relu":
143         A, activation_cache = relu(Z)
144
145     cache = (linear_cache, activation_cache)
146
147     return A, cache
148
149 def model_forward(X, parameters):
150     """
151     A function to implement the forward propagation for all layers
152     of a neural network
153
154     Arguments:
155     A_prev -- activation matrix from previous layer
156     W -- weight matrix of current layer
157     b -- bias vector of current layer
158
159     Returns:
160     A -- the output of the activation function
161     cache -- tuple containing (( A, W, b), (Z, )), stored as cache
162     for backpropagation computation
163     """
164
165     caches = []
166     A = X
167     # extracting the number of layers in the neural network
168     L = len(parameters) // 2
169
170     for l in range(1, L):
171         A_prev = A
172         A, cache = linear_activation_forward(A_prev, parameters['W'
173             + str(l)], parameters['b' + str(l)], "relu")
174         caches.append(cache)
175
176     AL, cache = linear_activation_forward(A, parameters['W' + str(L)
177         ], parameters['b' + str(L)], "sigmoid")
178     caches.append(cache)
179
180     return AL, caches
181
182 def has_converged(costs, window=5, threshold=0.001):
183     """
184     A function to check whether the cost computation has converged.

```

```

182 Arguments:
183 costs -- log of costs after each iteration
184 window -- the size of the window of iterations for which
           convergence needs to be checked
185 threshold -- the percentage change in the cost below which
              convergence can be declared
186
187 Returns:
188 True/False -- depending on whether the function has converged
189 """
190
191 if len(costs) < window + 1:
192     return False
193
194 recent = costs[-window:]
195 relative_changes = [abs(recent[i] - recent[i-1])/abs(recent[i-1]) for i in range(1, len(recent))]
196
197 return max(relative_changes) < threshold
198
199 def compute_cost(AL, Y):
200     """
201     A function to compute the cost of prediction while training.
202
203     Arguments:
204     AL -- probability vector corresponding to the predictions made
205     Y -- true label vector for the data
206
207     Returns:
208     cost -- cross-entropy cost
209     """
210
211     m = Y.shape[1]
212     epsilon = 1e-8
213     AL = np.clip(AL, epsilon, 1 - epsilon)
214     cost = -1/m * (np.dot(Y, np.log(AL).T) + np.dot(1-Y, np.log(1-AL).T))
215     cost = np.squeeze(cost)
216     return cost
217
218 def linear_backward(dZ, cache):
219     """
220     A function to implement the linear part of backward propagation
221     for a layer.
222
223     Arguments:
224     dZ -- gradient of the cost with respect to Z (output of the
           linear part of forward propagation)
           cache -- tuple containing (A_prev, W, b) coming from the forward
           propagation of the current layer

```

```

225
226 Returns:
227 dA_prev -- gradient of the cost with respect to activation (A)
           of the previous layer
228 dW -- gradient of the cost with respect to W of the current
           layer
229 db -- gradient of the cost with respect to b of the current
           layer
230 """
231
232 A_prev, W, b = cache
233 m = A_prev.shape[1]
234
235 dW = 1/m * np.dot(dZ, A_prev.T)
236
237 db = 1/m * np.sum(dZ, axis=1, keepdims=True)
238
239 dA_prev = np.dot(W.T, dZ)
240
241 return dA_prev, dW, db
242
243 def linear_activation_backward(dA, cache, activation):
244     """
245     A function to implement the backward propagation for a single
        layer.
246
247     Arguments:
248     dA - post-activation gradient for current layer
249     cache -- a tuple of values ((A, W, b), (Z, ))
250     activation -- the type of activation to be used for the current
        layer
251
252     Returns:
253     dA_prev - gradient of the cost with respect to the activation of
        the previous layer
254     dW -- gradient of the cost with respect to W of the current
        layer
255     db -- gradient of the cost with respect to b of the current
        layer
256     """
257
258     linear_cache, activation_cache = cache
259
260     if activation == "relu":
261         dZ = relu_backward(dA, activation_cache)
262
263     elif activation == "sigmoid":
264         dZ = sigmoid_backward(dA, activation_cache)
265
266     dA_prev, dW, db = linear_backward(dZ, linear_cache)

```

```

267
268     return dA_prev, dW, db
269
270 def model_backward(AL, Y, caches):
271     """
272     A function to implement the backward propagation for all the
273         layers of the neural network.
274
275     Arguments:
276     AL -- the probability vector of the predictions the network has
277         made
278     Y -- true label vector of the data
279     caches -- python list containing ((A, W, b), (Z, )) of every
280         layer of the neural network
281
282     Returns:
283     grads -- A dictionary with the gradients dA, dW, db for all
284         layers of the neural network
285     """
286
287     grads = {}
288     L = len(caches)
289     m = AL.shape[1]
290     Y = Y.reshape(AL.shape)
291
292     dAL = -(np.divide(Y, AL) - np.divide(1-Y, 1-AL))
293
294     current_cache = caches[L-1]
295     dA_prev_temp, dW_temp, db_temp = linear_activation_backward(dAL,
296         current_cache, "sigmoid")
297     grads["dA" + str(L-1)] = dA_prev_temp
298     grads["dW" + str(L)] = dW_temp
299     grads["db" + str(L)] = db_temp
300
301     for l in reversed(range(L-1)):
302
303         current_cache = caches[l]
304         dA_prev_temp, dW_temp, db_temp = linear_activation_backward(
305             grads["dA" + str(l+1)], current_cache, "relu")
306         grads["dA" + str(l)] = dA_prev_temp
307         grads["dW" + str(l+1)] = dW_temp
308         grads["db" + str(l+1)] = db_temp
309
310     return grads
311
312 def update_parameters(params, grads, learning_rate):
313     """
314     A function to update parameters using gradient descent
315
316     Arguments:

```

```

311     params -- dictionary containing the parameters W,b for all
312           layers of the neural network
313     grads -- A dictionary with the gradients dA, dW, db for all
314           layers of the neural network
315
316     Returns:
317     parameters -- dictionary containing updated parameters
318     """
319
320     parameters = copy.deepcopy(params)
321     L = len(parameters) // 2
322
323     for l in range(L):
324         parameters['W' + str(l+1)] = parameters["W" + str(l+1)] -
325             learning_rate * grads["dW" + str(l+1)]
326         parameters["b" + str(l+1)] = parameters['b' + str(l+1)] -
327             learning_rate * grads['db' + str(l+1)]
328
329     return parameters
330
331 def get_mini_batches(X, Y, batch_size=256, seed=0):
332     """
333     A function to separate the data into mini batches depending on
334     the batch size.
335
336     Arguments:
337     X -- input features of 'm' training examples
338     Y -- labels of 'm' training examples
339     batch_size -- number of batches that the data is to be divided
340     into
341     seed -- used as input to generate shuffled batches with each
342     iteration
343
344     Returns:
345     mini_batches - python list of tuples containing each X and Y
346     batch -> (X_batch, Y_batch)
347     """
348
349     np.random.seed(seed)
350     m = X.shape[1]
351     mini_batches = []
352
353     permutation = list(np.random.permutation(m))
354     X_shuffled = X[:, permutation]
355     Y_shuffled = Y[:, permutation]
356
357     num_complete_batches = m // batch_size
358
359     for i in range(num_complete_batches):
360         X_batch = X_shuffled[:, (i) * batch_size: (i+1) * batch_size]

```



```

353         ]
354         Y_batch = Y_shuffled[:, (i) * batch_size: (i+1) * batch_size]
355     ]
356     mini_batches.append((X_batch, Y_batch))
357
358     if m % batch_size != 0:
359         X_batch = X_shuffled[:, num_complete_batches * batch_size:]
360         Y_batch = Y_shuffled[:, num_complete_batches * batch_size:]
361         mini_batches.append((X_batch, Y_batch))
362
363     return mini_batches
364
365 def run_model(X, Y, layers_dims, learning_rate=0.075, num_iterations
366             = 3000, print_cost=False,
367             check_convergence=False, convergence_window=5,
368             convergence_threshold=0.001):
369
370     """
371     A function to run the neural network model.
372
373     Arguments:
374     X -- input features of 'm' training examples
375     Y -- labels of 'm' training examples
376     layers_dims -- list of the number of neurons in each layer.
377     learning_rate -- rate at which gradient descent is executed
378     num_iterations -- number of iterations that the model has to run
379                     for
380     print_cost -- boolean variable to toggle whether to print the
381                  cost after each 100th iteration
382     check-convergence -- boolean variable to toggle whether to check
383                        for convergence while running
384     convergence_window -- the size of the window of iterations for
385                          which convergence needs to be checked
386     convergence_threshold -- the percentage change in the cost below
387                             which convergence can be declared
388
389     Returns:
390     parameters -- dictionary of final weight and bias matrices for
391                  each layer of the neural network after training finishes
392     costs -- list of cost values logged after each 100th iteration
393     """
394
395     costs = []
396
397     parameters = initialize_parameters(layers_dims)
398
399     for i in range(0, num_iterations):
400
401         AL, caches = model_forward(X, parameters)
402         cost = compute_cost(AL, Y)

```

```

393     grads = model_backward(AL, Y, caches)
394     parameters = update_parameters(parameters, grads,
395                                     learning_rate)
396
397     if print_cost and (i % 100 == 0 or i == num_iterations - 1):
398         print("Cost after iteration {}: {}".format(i, np.squeeze
399             (cost)))
400     if i % 100 == 0:
401         costs.append(cost)
402
403     if check_convergence and i % 100 == 0:
404         if has_converged(costs, window=convergence_window,
405             threshold=convergence_threshold):
406             print(f"Converged at iteration {i} (relative change
407                 < {convergence_threshold})")
408             break
409
410     return parameters, costs
411
412 def run_model_epochs(X, Y, layers_dims, learning_rate=0.075,
413                     num_epochs = 100, batch_size=256, print_cost=False,
414                     check_convergence=False, convergence_window=5,
415                     convergence_threshold=0.001):
416
417     """
418     A function to run the neural network model with the mini-
419     batching.
420
421     Arguments:
422     X -- input features of 'm' training examples
423     Y -- labels of 'm' training examples
424     layers_dims -- list of the number of neurons in each layer.
425     learning_rate -- rate at which gradient descent is executed
426     num_epochs -- number of iterations for which the model has to
427     run
428     batch_size -- number of batches that the data is to be divided
429     into
430     print_cost -- boolean variable to toggle whether to print the
431     cost after each 100th iteration
432     check-convergence -- boolean variable to toggle whether to check
433     for convergence while running
434     convergence_window -- the size of the window of iterations for
435     which convergence needs to be checked
436     convergence_threshold -- the percentage change in the cost below
437     which convergence can be declared
438
439     Returns:
440     parameters -- dictionary of final weight and bias matrices for
441     each layer of the neural network after training finishes
442     costs -- list of cost values logged after each 100th iteration

```

```

429     """
430
431     costs = []
432
433     parameters = initialize_parameters(layers_dims)
434
435     for i in range(num_epochs):
436
437         mini_batches = get_mini_batches(X, Y, batch_size, seed=i)
438         epoch_cost = 0
439
440         for X_batch, Y_batch in mini_batches:
441             AL, caches = model_forward(X_batch, parameters)
442
443             batch_cost = compute_cost(AL, Y_batch)
444
445             grads = model_backward(AL, Y_batch, caches)
446             parameters = update_parameters(parameters, grads,
447                                             learning_rate)
448
449             epoch_cost += batch_cost * X_batch.shape[1] / X.shape[1]
450
451         if print_cost and (i % 10 == 0 or i == num_epochs - 1):
452             print("Cost after iteration {}: {}".format(i, np.squeeze(
453                 epoch_cost)))
454         if i % 10 == 0:
455             costs.append(epoch_cost)
456
457         if check_convergence and i % 10 == 0:
458             if has_converged(costs, window=convergence_window,
459                             threshold=convergence_threshold):
460                 print(f"Converged at iteration {i} (relative change
461                     < {convergence_threshold})")
462                 break
463
464     return parameters, costs
465
466 def predict(X, y, parameters):
467     """
468     A function to run the neural network model.
469
470     Arguments:
471     X -- input features of 'm' training examples
472     Y -- labels of 'm' training examples
473     parameters -- dictionary of final weight and bias matrices for
474                  each layer of the neural network after training finishes
475
476     Returns:
477     p -- the probability matrix of predictions made using the model
478     np.sum((p==y)/m) -- the accuracy of the predictions of the model

```

```

474     """
475
476     m = X.shape[1]
477     n = len(parameters) // 2
478     p = np.zeros((1, m))
479
480     probas, caches = model_forward(X, parameters)
481
482     for i in range(0, probas.shape[1]):
483         if probas[0, i] > 0.5:
484             p[0, i] = 1
485         else:
486             p[0, i] = 0
487
488     print("Accuracy: " + str(np.sum((p==y)/m)))
489     return p, np.sum((p==y)/m)

```